
Android Application Penetration Testing
DIVA (Damn Insecure and Vulnerable App)
Lab Documentation
Index
1 Introduction 2
2 Lab Environment & Tools 3
3 Lab 1: Insecure Logging 4
4 Lab 2: Insecure Data Storage 6
5 Lab 3: SQL Injection 8
6 Lab 4: Insecure Authentication 10
7 Lab 5: Input Validation Issues 12
8 Lab 6: Access Control Issues – Part 1 14
9 Lab 7: Access Control Issues – Part 2 16
10 Lab 8: Access Control Issues – Part 3 18
11 Summary of Findings 20
12 Conclusion 21
10:30
DIVA
Damn Insecure and Vulnerable App

ANDROID APPLICATION PENTESTING
DIVA LAB DOCUMENTATION

Practical Learning Record — DIVA (Damn Insecure and Vulnerable App)

Environment: Windows, Android Emulator, ADB, JADX

1. Introduction

This document records Android application security exercises completed using DIVA. Topics include static analysis, Android logs, local storage, SQL injection, WebView URI handling, Intents, and Content Providers. Tests were performed against the intentionally vulnerable local DIVA app; credentials and notes are lab data.

2. Environment and Tools

- Windows host and Android Emulator (emulator-5554).
- DIVA package: jakhar.aseem.diva
- JADX: inspect/decompile application code.
- ADB: launch components, inspect logs, query providers, and interact with emulator.
- SQLite viewer: inspect extracted lab database.
- Frida, Objection, and Burp Suite were not used in these completed labs.

3. Lab 1 — Insecure Logging

Objective

Check whether sensitive user input is written to Android logs.

Analysis and test

LogActivity called processCC(). When processing failed, its exception handler logged the supplied card input.

Command: adb logcat -s diva-log:E

Observation

Logcat showed the synthetic card value entered in the lab, demonstrating sensitive-data exposure.

Impact and remediation

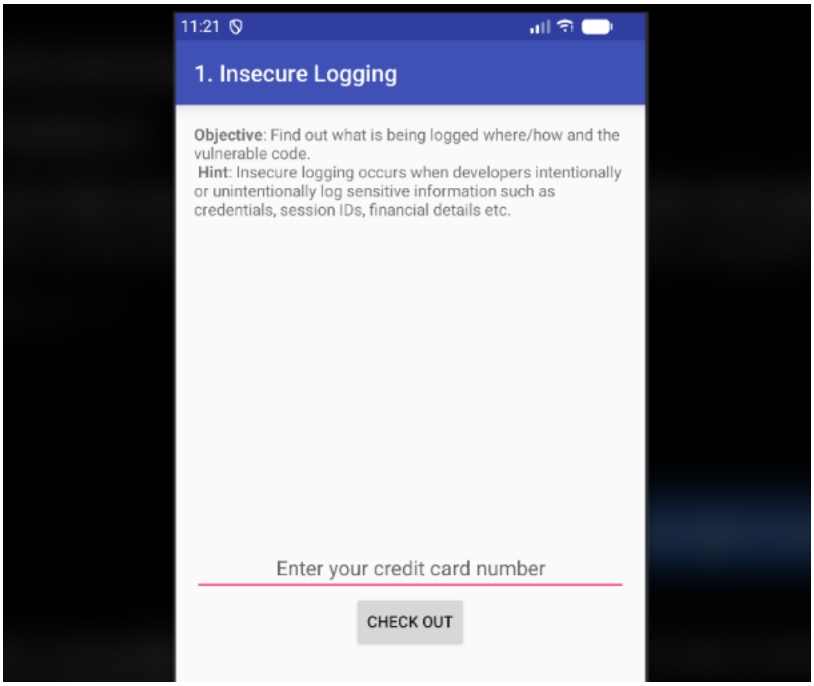
Secrets in logs may be exposed through debugging or diagnostics. Do not log card data, passwords, tokens, or other secrets; redact sensitive values.

Context and learning

This exercise showed me that information can leak even when it is not displayed on the screen or saved in a file. Android logs are often used by developers to troubleshoot errors, so logging sensitive input may seem convenient during development. However, diagnostic

output can become another place where private data is exposed. I used a synthetic card number in this lab and checked the relevant log output to confirm what the application recorded.

[SCREENSHOT AREA — Lab 1 — logcat output]




```
49      public void checkout(View view) {
50          EditText cctxt = (EditText) findViewById(R.id.ccText);
            try {
54              processCC(cctxt.getText().toString());
            } catch (RuntimeException e) {
56              Log.e("diva-log", "Error while processing transaction with credit card: " + cctxt.getText().toString());
57              Toast.makeText(this, "An error occured. Please try again later", 0).show();
            }
        }
```

```
>adb logcat -c

adb logcat -s diva-log:E
--------- beginning of main
11-06 11:28:06.348  4635  4635 E diva-log: Error while processing transaction with credit card: 41
11111111111111
11-06 12:12:10.750  4635  4635 E diva-log: Error while processing transaction with credit card: 41
11111111111111
^C
```

4. Lab 2 — Insecure Data Storage Part 1 (SharedPreferences)

Objective
Check whether credentials are stored in plaintext preferences.

Test
After entering synthetic credentials, inspect preferences:

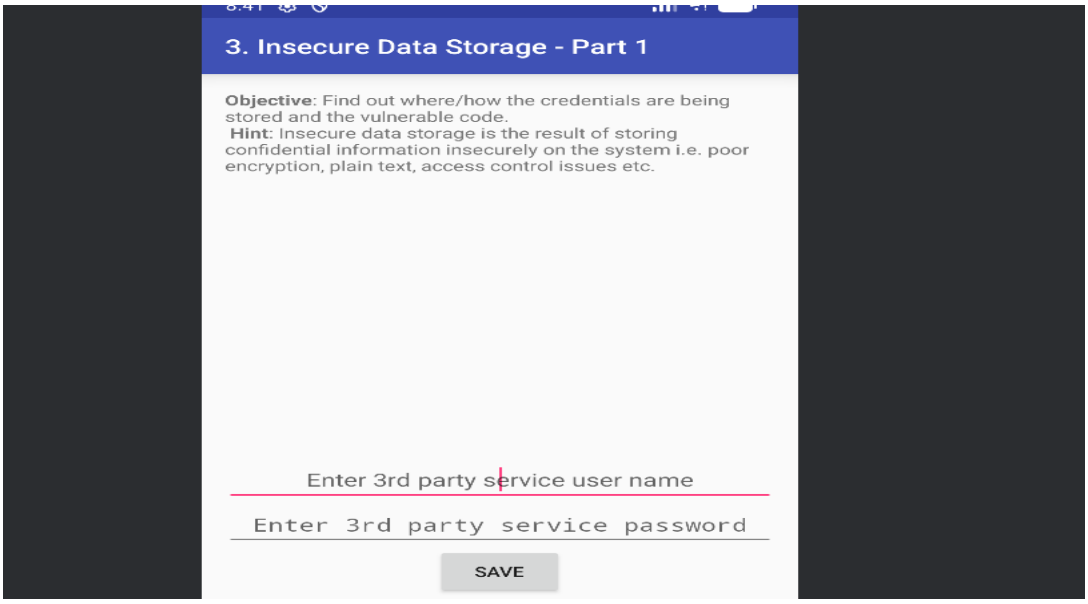
adb shell run-as jakhar.aseem.diva cat shared_prefs/jakhar.aseem.diva_preferences.xml

Observation
The username/password values were visible in plaintext XML.

Remediation
App-private storage is not encryption. Avoid storing passwords; use secure authentication and Keystore-backed encryption for secrets that must persist.

Context and learning
SharedPreferences is commonly used by Android applications to keep small values such as settings and preferences. It is easy to use, but the fact that the file is stored in the app's private directory should not be mistaken for encryption. In this exercise, I inspected the XML file directly and saw that the credentials were readable. This helped me understand the difference between restricting access to a file and protecting the contents of that file.

[SCREENSHOT AREA — Lab 2 — SharedPreferences XML]



```
tivity  NotesProvider  LogActivity  InsecureDataStorage1Activity

package jakhar.aseem.diva;

import android.content.SharedPreferences;
import android.os.Bundle;
import android.preference.PreferenceManager;
import android.support.v7.app.AppCompatActivity;
import android.view.View;
import android.widget.EditText;
import android.widget.Toast;

/* JADX INFO: loaded from: classes.dex */
44  public class InsecureDataStorage1Activity extends AppCompatActivity {
        @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.FragmentActivity
45      protected void onCreate(Bundle savedInstanceState) {
46          super.onCreate(savedInstanceState);
47          setContentView(R.layout.activity_insecure_data_storage1);
        }

50      public void saveCredentials(View view) {
51          SharedPreferences spref = PreferenceManager.getDefaultSharedPreferences(this);
52          SharedPreferences.Editor spedit = spref.edit();
53          EditText usr = (EditText) findViewById(R.id.ids1Usr);
54          EditText pwd = (EditText) findViewById(R.id.ids1Pwd);
56          spedit.putString("user", usr.getText().toString());
57          spedit.putString("password", pwd.getText().toString());
58          spedit.commit();
60          Toast.makeText(this, "3rd party credentials saved successfully!", 0).show();
        }
    }
```

```
[redacted]>adb shell run-as jakhar.aseem.diva ls shared_prefs
jakhar.aseem.diva_preferences.xml
```

```
[redacted]>adb shell run-as jakhar.aseem.diva cat shared_prefs/jakhar.aseem.diva_pre
ferences.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="password">test@123</string>
    <string name="user">test</string>
</map>
```

5. Lab 3 — Insecure Data Storage Part 2 (SQLite)

Objective
Inspect local SQLite data and identify plaintext credentials and unsafe SQL construction.

Analysis
The lab created database ids2 and table myuser(user,password). User input was concatenated into SQL.

Test
Extract database: cmd.exe /c "adb exec-out run-as jakhar.aseem.diva cat databases/ids2 > ids2"

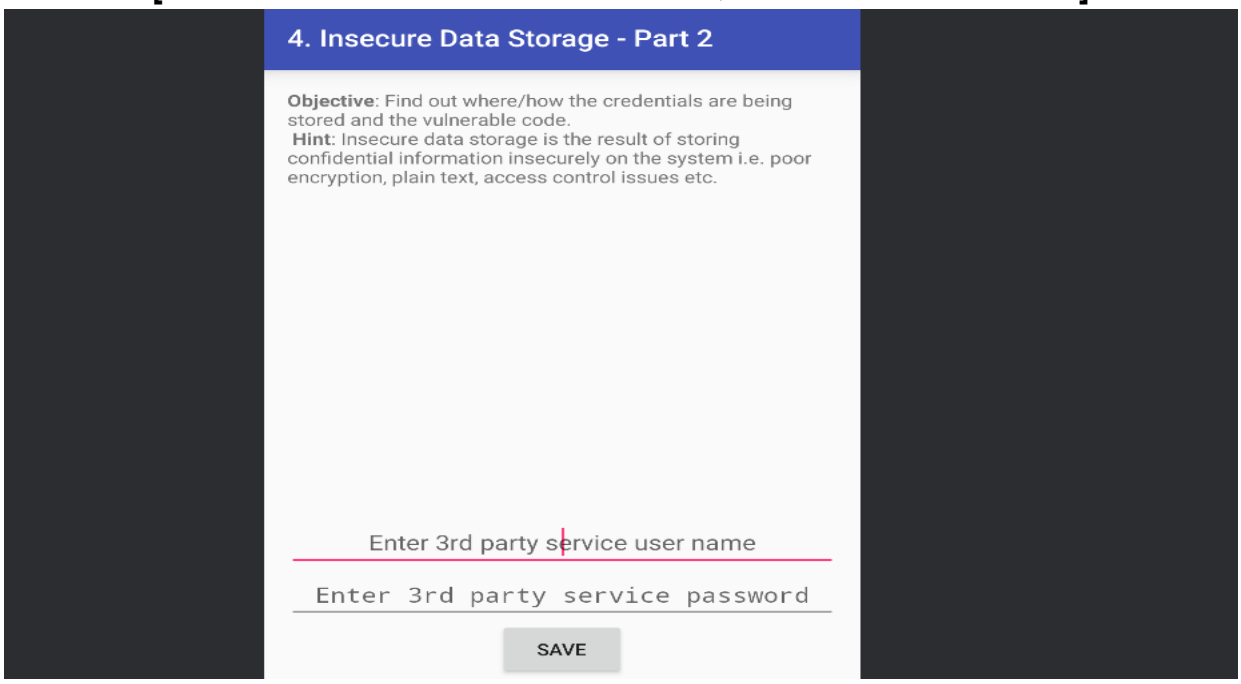
Open the extracted database in a SQLite viewer.

Observation
Credentials were stored in plaintext; SQL concatenation created injection risk.

Remediation
Use parameterized SQL statements and avoid storing passwords.

Context and learning
SQLite is a normal and useful choice for storing structured data on a device, but sensitive information still needs appropriate protection. This lab combined two issues: credentials were stored in readable form, and the application built a SQL statement by joining strings. Looking at the database helped verify the storage issue, while reviewing the code made the injection risk clear. These are separate weaknesses and should be addressed separately: protect sensitive data and use parameterized queries.

[SCREENSHOT AREA — Lab 3 — SQLite database contents]



```
LogActivity | InsecureDataStorage1Activity | InsecureDataStorage2Activity

package jakhar.aseem.diva;

import android.database.sqlite.SQLiteDatabase;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.util.Log;
import android.view.View;
import android.widget.EditText;
import android.widget.Toast;

/* JADX INFO: loaded from: classes.dex */
49  public class InsecureDataStorage2Activity extends AppCompatActivity {
        private SQLiteDatabase mDB;

        @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.FragmentActivity, android.support.v4.app.BaseFragmentActivityDon
50      protected void onCreate(Bundle savedInstanceState) {
51          super.onCreate(savedInstanceState);
            try {
53              SQLiteDatabase sQLiteDatabaseOpenOrCreateDatabase = openOrCreateDatabase("ids2", 0, null);
                this.mDB = sQLiteDatabaseOpenOrCreateDatabase;
54              sQLiteDatabaseOpenOrCreateDatabase.execSQL("CREATE TABLE IF NOT EXISTS myuser(user VARCHAR, password VARCHAR);");
            } catch (Exception e) {
57              Log.d("Diva", "Error occurred while creating database: " + e.getMessage());
            }
60          setContentView(R.layout.activity_insecure_data_storage2);
        }

63      public void saveCredentials(View view) {
64          EditText usr = (EditText) findViewById(R.id.ids2Usr);
65          EditText pwd = (EditText) findViewById(R.id.ids2Pwd);
            try {
67              this.mDB.execSQL("INSERT INTO myuser VALUES ('" + usr.getText().toString() + "', '" + pwd.getText().toString() + "');");
68              this.mDB.close();
            } catch (Exception e) {
71              Log.d("Diva", "Error occurred while inserting into database: " + e.getMessage());
            }
73          Toast.makeText(this, "3rd party credentials saved successfully!", 0).show();
        }
    }
```

```
>adb shell run-as jakhar.aseem.diva ls
cache
code_cache
databases
shared_prefs

>adb shell run-as jakhar.aseem.diva pwd
/data/user/0/jakhar.aseem.diva

>adb shell run-as jakhar.aseem.diva ls databases
divanotes.db
divanotes.db-journal
ids2
ids2-journal
```

Dumped the database in local machine.

```
>cmd.exe /c "adb exec-out run-as jakhar.aseem.diva cat databases/ids2 > id
s2"
```

6. Lab 4 — Insecure Data Storage Part 3(External Storage)

Objective
Check whether sensitive data is stored in shared external storage.

Test
The lab wrote credentials to /storage/emulated/0/.uinfo.txt. Read it using:

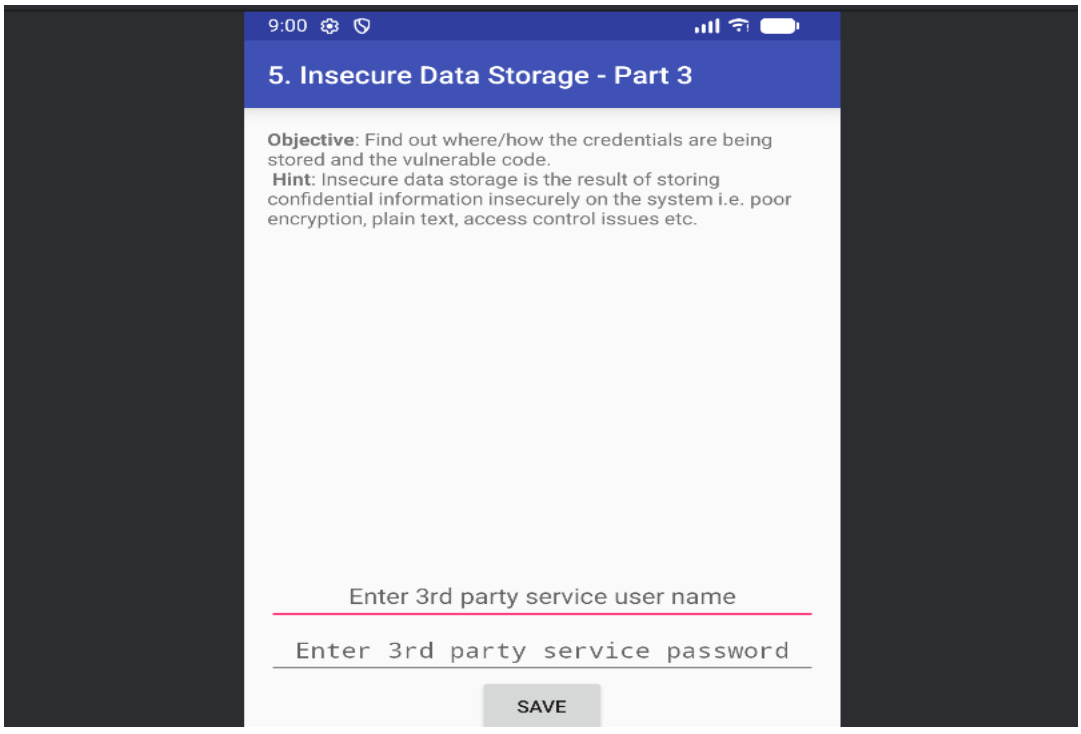
adb shell cat /storage/emulated/0/.uinfo.txt

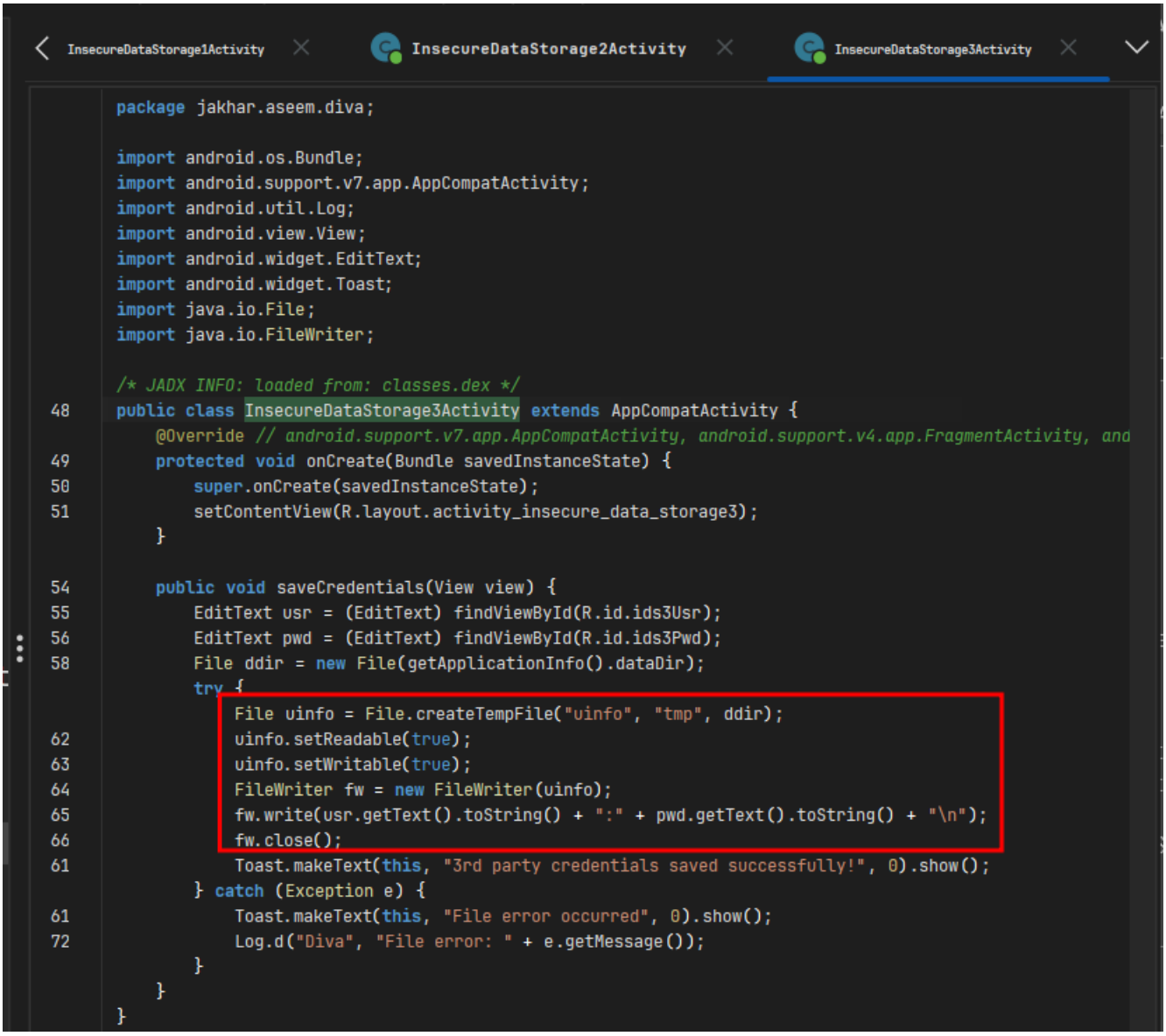
Observation
The file contained synthetic username/password in plaintext. A dot-prefixed filename does not protect it.

Remediation
Do not store secrets in shared external storage. Use app-private storage and appropriate encryption.

Context and learning
External or shared storage is intended for files that may be accessible outside the application's private area. A filename beginning with a dot may make a file less visible in some file browsers, but it does not provide access control or encryption. By reading the file from the emulator, I confirmed that the data was exposed as ordinary text. The key lesson is to choose storage based on the sensitivity of the data, not on the filename or its apparent visibility.

[SCREENSHOT AREA — Lab 4 — external storage file]






```
[redacted]>adb shell run-as jakhar.aseem.diva ls
cache
code_cache
databases
shared_prefs
uinfo3025176840272416968tmp
uinfo4855637161377050427tmp

[redacted]>adb shell run-as jakhar.aseem.diva cat uinfo3025176840272416968tmp
test2:test2@445
```

7. Lab 5 — Input Validation Part 1 (SQL Injection)

Objective
Test whether search input is concatenated into SQL.

Vulnerable pattern
SELECT * FROM sqliuser WHERE user = ' + user_input + '

Test
Entered payload only (not the full SQL statement):

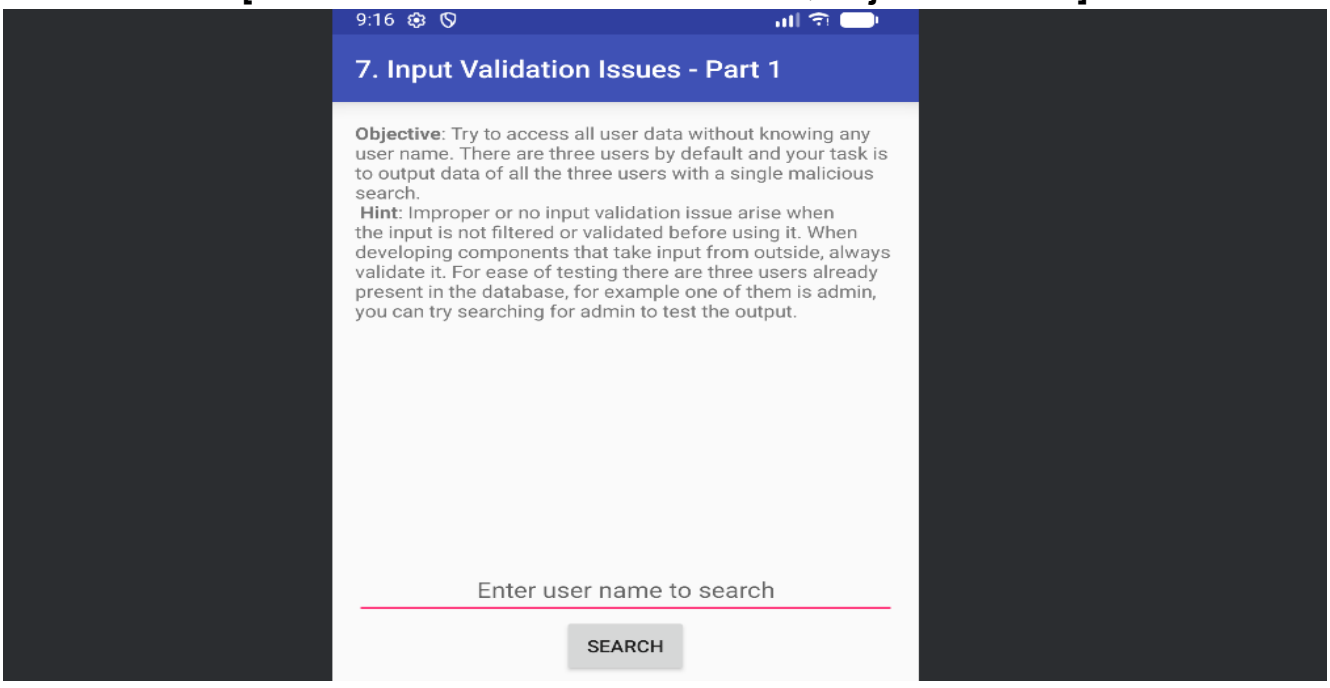
' OR '1'='1

Observation
The injected condition altered the WHERE clause and returned a matching record; the UI showed the first result.

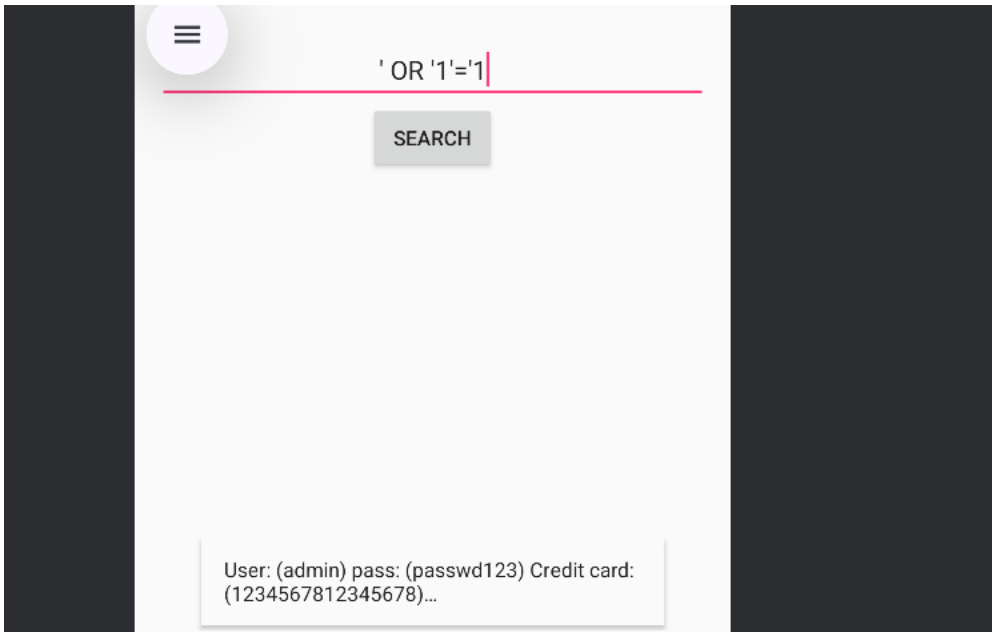
Remediation
Use parameterized SQL, e.g. rawQuery("SELECT * FROM sqliuser WHERE user = ?", new String[]{input}).

Context and learning
The important part of this test was understanding how the input changed the meaning of the SQL query. The application expected a username, but because it inserted the input directly into the query text, the input could be interpreted as SQL syntax instead of just data. I entered the payload into the application's search field rather than submitting a complete SQL statement. The result demonstrated why user input must remain separate from query structure.

[SCREENSHOT AREA — Lab 5 — SQL injection result]



```
66      public void search(View view) {
67          EditText srchtxt = (EditText) findViewById(R.id.ivi1search);
            try {
70              Cursor cr = this.mDB.rawQuery("SELECT * FROM sqliuser WHERE user = '" + srchtxt.getText().toString() + "'", null);
71              StringBuilder strb = new StringBuilder("");
                if (cr != null && cr.getCount() > 0) {
73                  cr.moveToFirst();
77                  do {
72                      strb.append("User: (" + cr.getString(0) + ") pass: (" + cr.getString(1) + ") Credit card: (" + cr.getString(2) + ")\n");
77                  } while (cr.moveToNext());
                } else {
80                  strb.append("User: (" + srchtxt.getText().toString() + ") not found");
                }
72              Toast.makeText(this, strb.toString(), 0).show();
            } catch (Exception e) {
85              Log.d("Diva-sqli", "Error occurred while searching in database: " + e.getMessage());
            }
        }
    }
```



Trails lead means other users data is been reflected after admin.

8. Lab 6 — Input Validation Part 2 (URI Scheme / WebView)

Objective
Review user-controlled URL handling in a WebView.

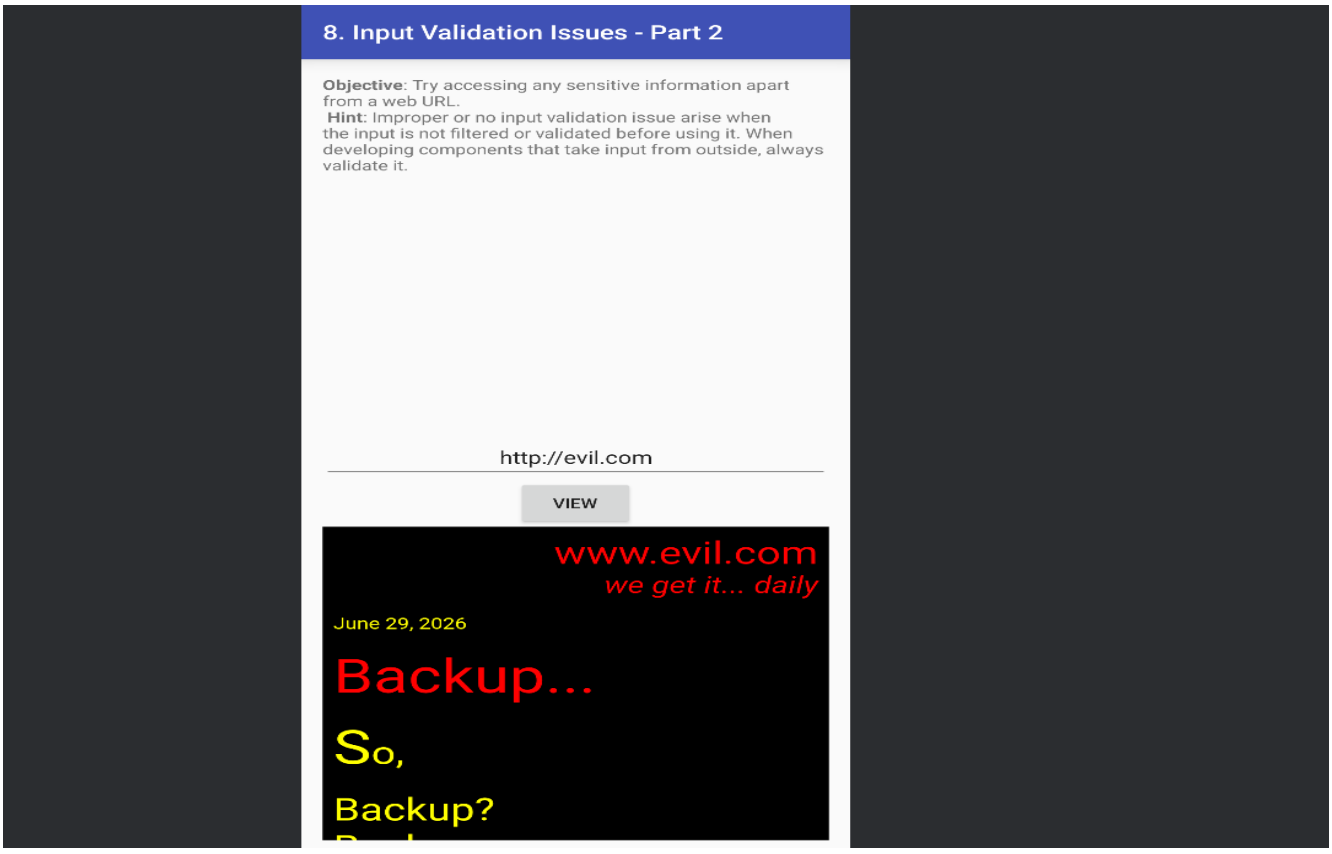
Analysis
The Activity enabled JavaScript and passed user-provided text to WebView.loadUrl().

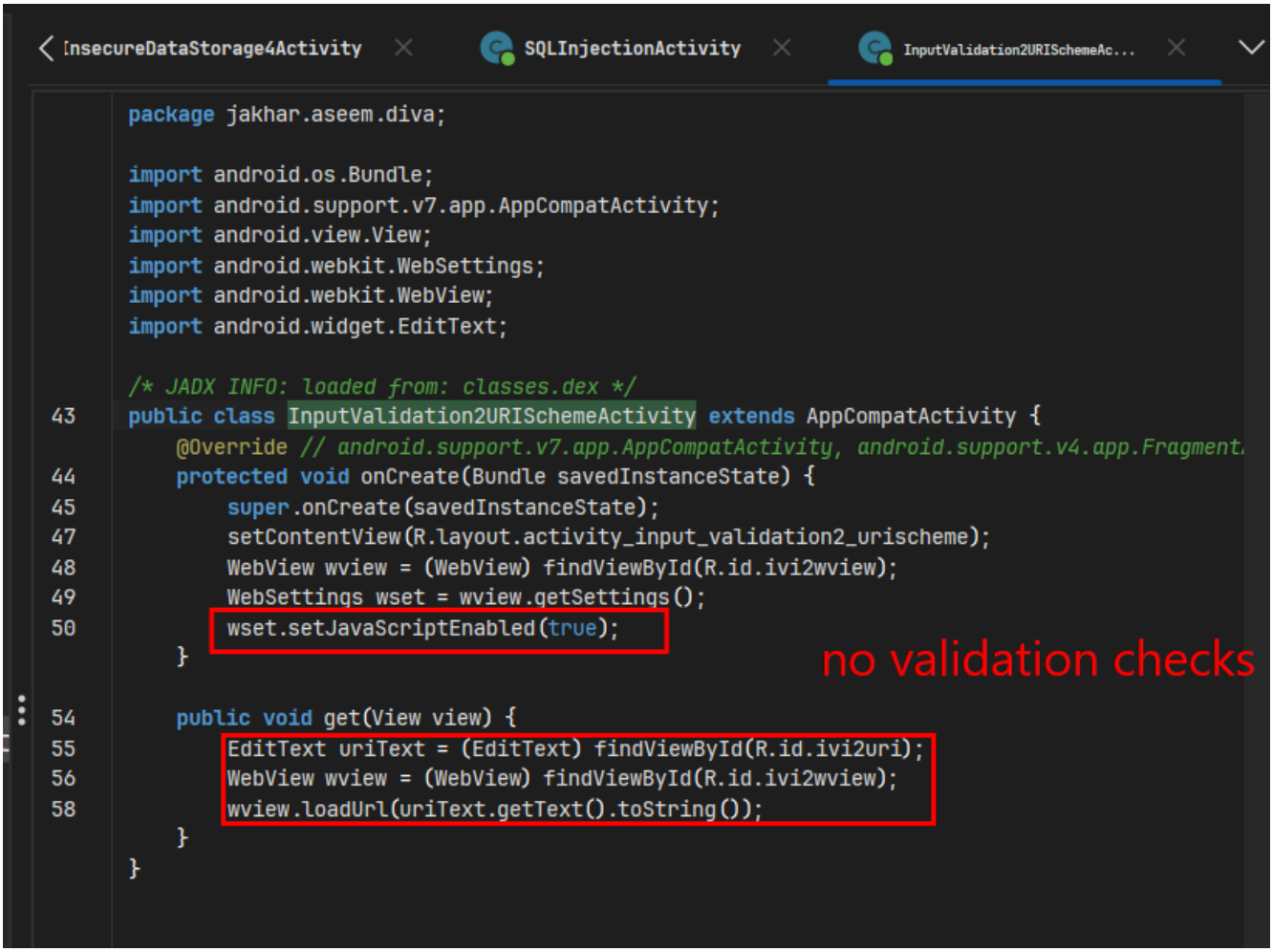
Test and observation
Tested URLs and a javascript: URI. The alert did not execute in the observed environment; the WebView appeared to accept HTTP-based URLs.

Remediation
Allowlist schemes/hosts, avoid arbitrary URL loading, disable JavaScript if unnecessary, and validate navigation.

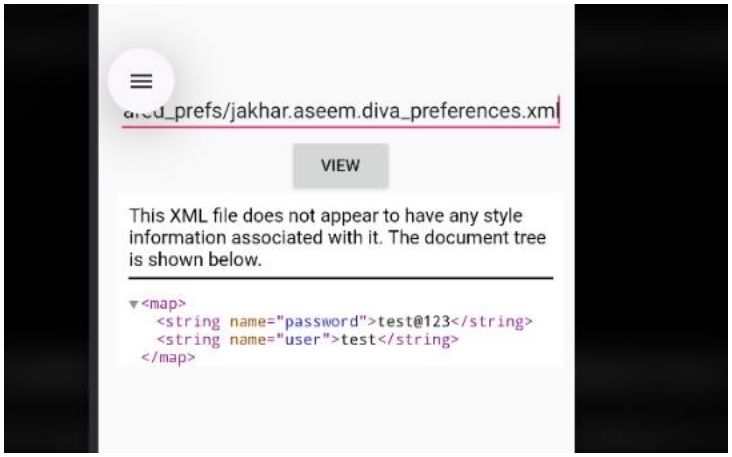
Context and learning
WebViews let an Android application display web content, but they also introduce a boundary between application code and content supplied through URLs. JavaScript support and URL loading behavior need to be considered together. In my test, the JavaScript alert did not execute, and the observed behavior appeared to favor HTTP-based URLs. I recorded this as an observation rather than claiming a successful exploit. A secure implementation should explicitly validate the schemes and destinations it intends to support.

[SCREENSHOT AREA — Lab 6 — WebView URL and behavior]





source code file view:
file:///data/data/jakhar.aseem.diva/shared_prefs/jakhar.aseem.diva_preferences.xml



9. Lab 7 — Access Control Part 1 (Intent-Based Activity Access)

Objective
Investigate whether an Activity showing credential information can be launched through an Intent.

Test
adb shell am start -a jakhar.aseem.diva.action.VIEW_CREDS

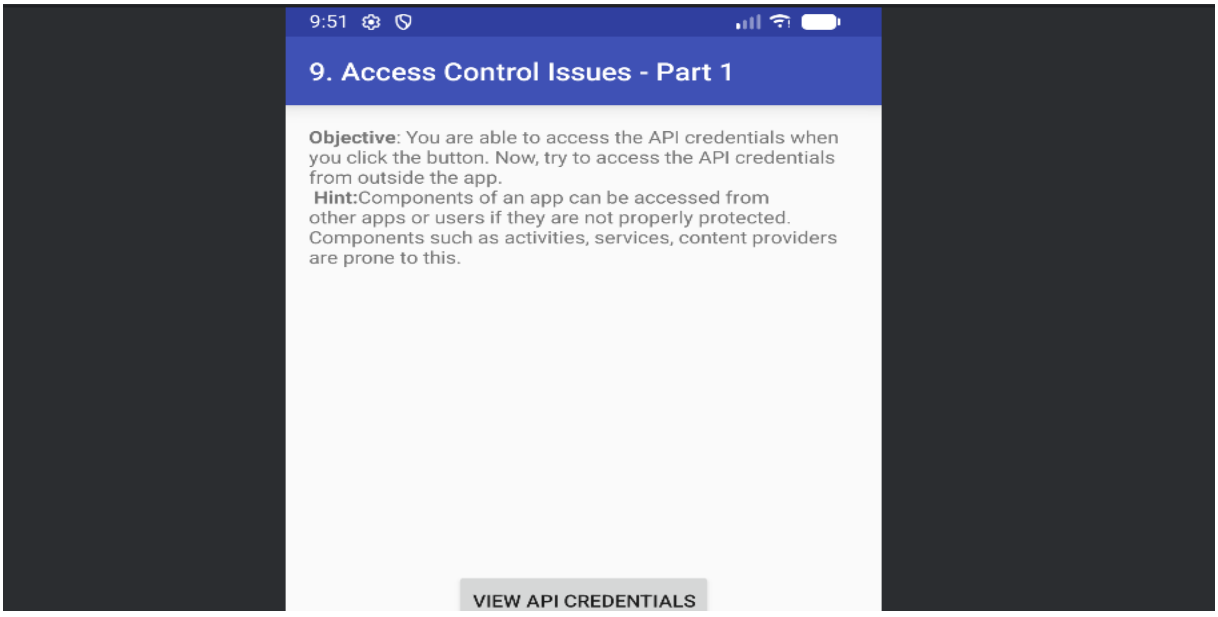
Observation
The Activity launch request succeeded. ADB launch output does not automatically return Activity UI contents to the terminal.

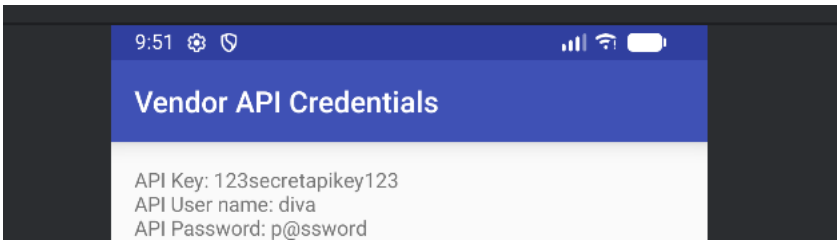
Assessment note
Launch alone does not prove third-party access. Check exported status, permissions, and actual data exposure.

Remediation
Set components non-exported when external access is unnecessary; otherwise enforce permissions and authorization.

Context and learning
Android components communicate through Intents, which can make application features available to other components and, in some configurations, other applications. This lab helped me distinguish between being able to request that an Activity launch and being able to access sensitive information inside it. The successful ADB launch was evidence that the Intent could resolve and start the Activity, but it was not sufficient by itself to prove a complete data-exposure vulnerability. Component export settings and permissions are important parts of the assessment.

[SCREENSHOT AREA — Lab 7 — Activity launch / emulator screen]






```
lidation2URISchemeAc...   AccessControl1Activity   InputValidation3Activity

package jakhar.aseem.diva;

import android.content.Intent;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.util.Log;
import android.view.View;
import android.widget.Toast;

/* JADX INFO: loaded from: classes.dex */
43  public class AccessControl1Activity extends AppCompatActivity {
        @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.Fragment
44      protected void onCreate(Bundle savedInstanceState) {
45          super.onCreate(savedInstanceState);
46          setContentView(R.layout.activity_access_control1);
        }

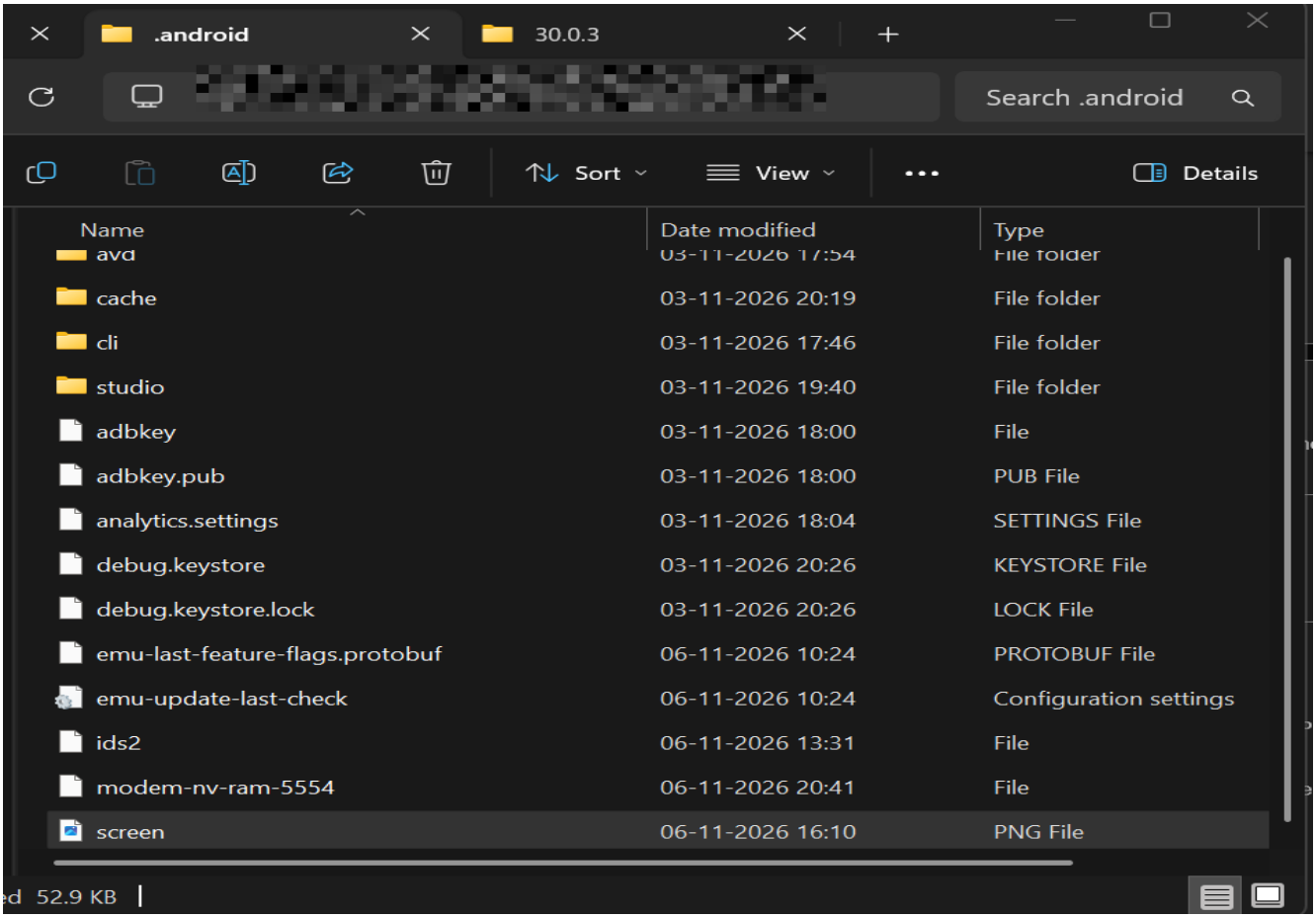
50      public void viewAPICredentials(View view) {
51          Intent i = new Intent();
52          i.setAction("jakhar.aseem.diva.action.VIEW_CREDS");
54          if (i.resolveActivity(getPackageManager()) != null) {
55              startActivity(i);
            } else {
58              Toast.makeText(this, "Error while getting API details", 0).show();
59              Log.e("Diva-aci1", "Couldn't resolve the Intent VIEW_CREDS to our activity");
            }
        }
    }
```

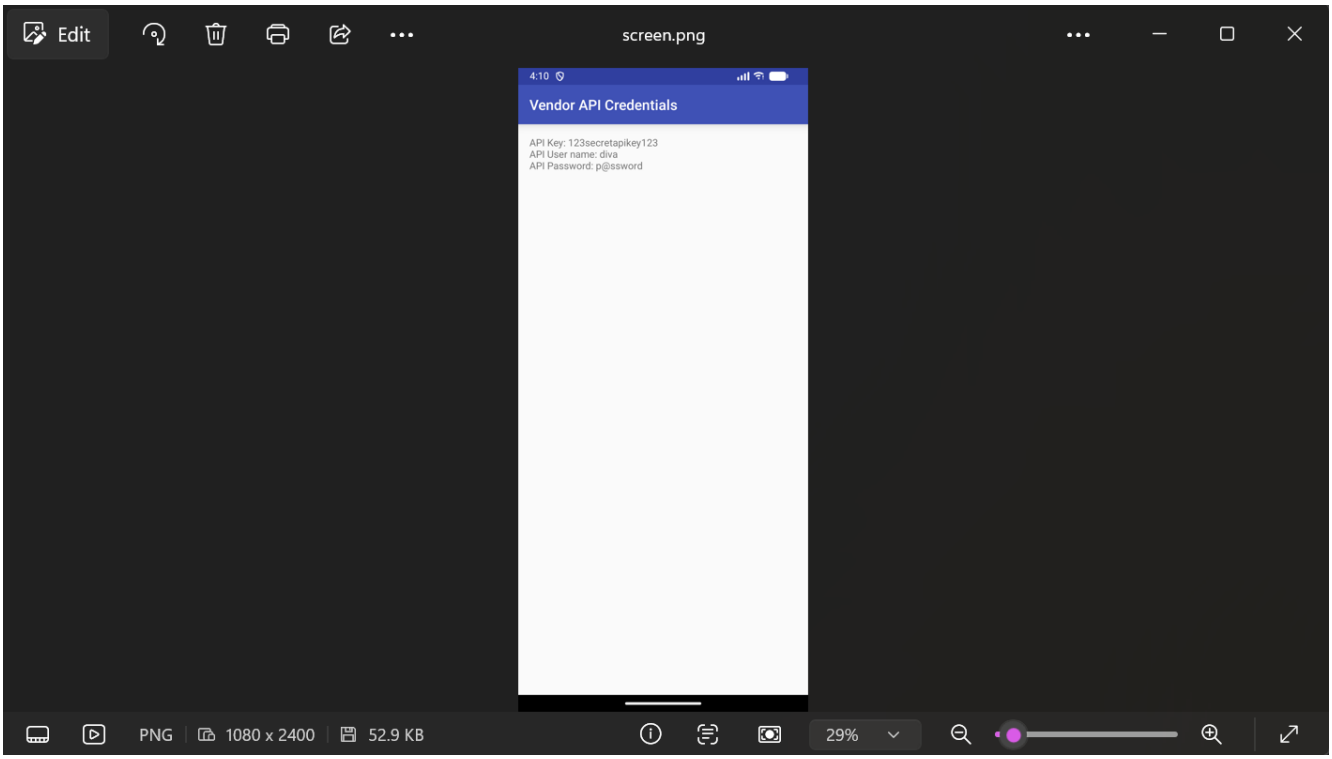
```
>adb shell am start -a jakhar.aseem.diva.action.VIEW_CREDS
Starting: Intent { act=jakhar.aseem.diva.action.VIEW_CREDS }

-adb shell screencap -p /sdcard/screen.png

-adb pull /sdcard/screen.png .
/sdcard/screen.png: 1 file pulled, ...d. 6.5 MB/s (54184 bytes in 0.008s)
```

Initiating the action external “VIEW_CREDS”, my plan was to retrieve the page data by screenshotting the page and pull that image in our local device.


.android
30.0.3
Search .android
Sort
View
Details
Name
Date modified
Type
avd
cache
03-11-2026 20:19
File folder
cli
03-11-2026 17:46
File folder
studio
03-11-2026 19:40
File folder
adbkey
03-11-2026 18:00
File
adbkey.pub
03-11-2026 18:00
PUB File
analytics.settings
03-11-2026 18:04
SETTINGS File
debug.keystore
03-11-2026 20:26
KEYSTORE File
debug.keystore.lock
03-11-2026 20:26
LOCK File
emu-last-feature-flags.protobuf
06-11-2026 10:24
PROTOBUF File
emu-update-last-check
06-11-2026 10:24
Configuration settings
ids2
06-11-2026 13:31
File
modem-nv-ram-5554
06-11-2026 20:41
File
screen
06-11-2026 16:10
PNG File
52.9 KB


Edit
screen.png
Vendor API Credentials
API Key: 123secretapikey123
API User name: diva
API Password: p@ssword
PNG
1080 x 2400
52.9 KB
29%

10. Lab 8 — Access Control Part 2 (PIN Bypass via Intent Extra)

Objective
Test whether an Intent extra can bypass PIN registration.

Analysis
Caller supplied Boolean extra check_pin. Receiver displayed credentials when false.

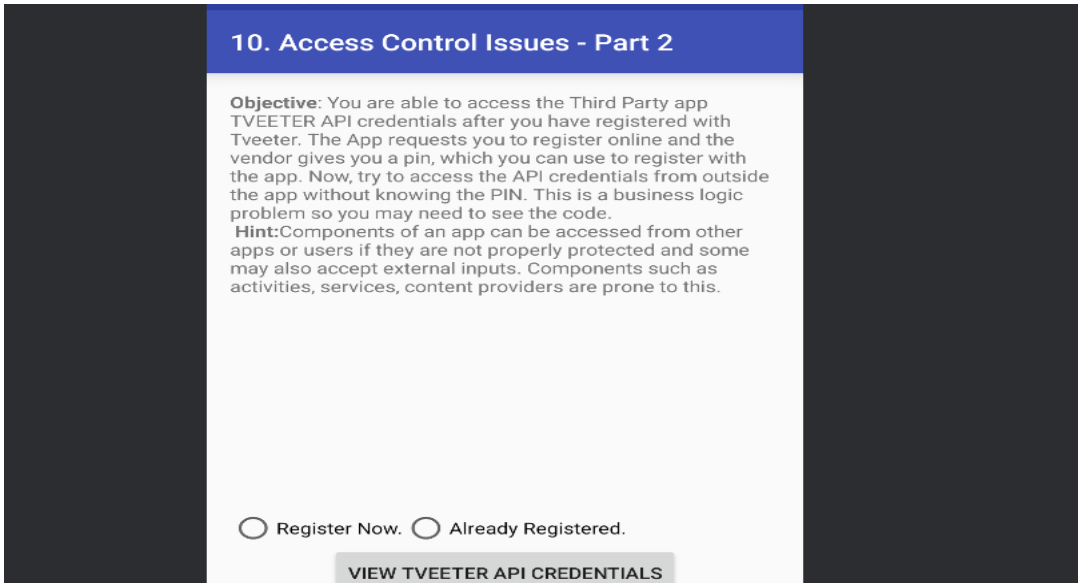
Test command
adb shell am start -n jakhar.aseem.diva/.APICreds2Activity --ez check_pin false

Observation
The app displayed lab API credentials without the PIN/registration flow.

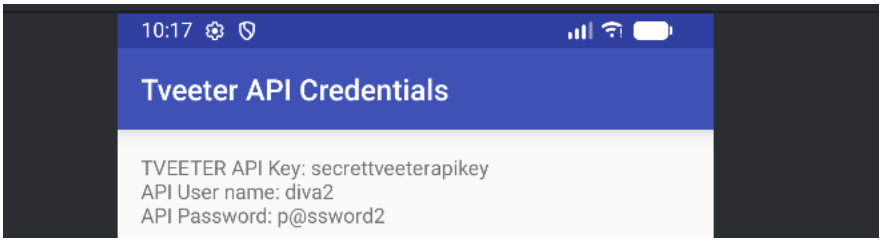
Impact and remediation
The app trusted caller-controlled input as an authentication decision. Never trust Intent extras for authentication; enforce authorization in the trusted component/backend and avoid embedding secrets in APKs.

Context and learning
Intent extras are values supplied by the component sending the Intent. They can be useful for passing information, but they should not be treated as proof that a user has authenticated. In this lab, the receiving Activity trusted a Boolean value named check_pin and displayed the credentials when the value was false. By controlling that extra through ADB, I demonstrated that the PIN decision was being made using untrusted input. Authentication and authorization decisions must be made by trusted code.

[SCREENSHOT AREA — Lab 8 — credential display after bypass]



Already registered user can view their creds



ntrol1Activity | InputValidation3Activity | AccessControl2Activity

```
package jakhar.aseem.diva;

import android.content.Intent;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.util.Log;
import android.view.View;
import android.widget.RadioButton;
import android.widget.Toast;

/* JADX INFO: loaded from: classes.dex */
44  public class AccessControl2Activity extends AppCompatActivity {
        @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.Fragment
45      protected void onCreate(Bundle savedInstanceState) {
46          super.onCreate(savedInstanceState);
47          setContentView(R.layout.activity_access_control2);
        }

51      public void viewAPICredentials(View view) {
52          RadioButton rbregnow = (RadioButton) findViewById(R.id.aci2rbregnow);
53          Intent i = new Intent();
54          boolean chk_pin = rbregnow.isChecked();
57          i.setAction("jakhar.aseem.diva.action.VIEW_CREDS2");
58          i.putExtra(getString(R.string.chk_pin), chk_pin);
60          if (i.resolveActivity(getPackageManager()) != null) {
61              startActivity(i);
            } else {
64              Toast.makeText(this, "Error while getting Tveeter API details", 0).show();
65              Log.e("Diva-aci1", "Couldn't resolve the Intent VIEW_CREDS2 to our activity");
            }
        }
    }
```

To access the contents of in APiCreds2Activity file .The application trusted a caller-controlled Intent extra to decide whether authentication had occurred.

alidation3Activity | APICreds2Activity | AccessControl2Activity

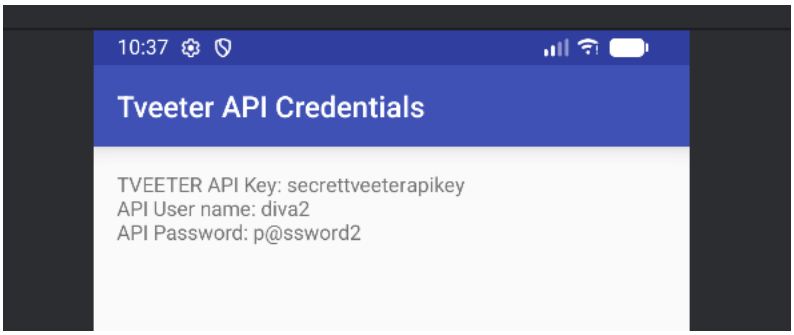
```
package jakhar.aseem.diva;

import android.content.Intent;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;

/* JADX INFO: loaded from: classes.dex */
46  public class APICreds2Activity extends AppCompatActivity {
        @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.FragmentActivity, android.support.v4.app.BaseFragmentActivityDa
47      protected void onCreate(Bundle savedInstanceState) {
48          super.onCreate(savedInstanceState);
49          setContentView(R.layout.activity_apicreds2);
50          TextView apicview = (TextView) findViewById(R.id.apic2TextView);
51          EditText pintext = (EditText) findViewById(R.id.aci2pinText);
52          Button vbutton = (Button) findViewById(R.id.aci2button);
53          Intent i = getIntent();
54          boolean bcheck = i.getBooleanExtra(getString(R.string.chk_pin), true);
56          if (!bcheck) {
61              apicview.setText("TVEETER API Key: secrettveeterapikey\nAPI User name: diva2\nAPI Password: p@ssword2");
68              return;
            }
64          apicview.setText("Register yourself at http://payatu.com to get your PIN and then login with that PIN!");
65          pintext.setVisibility(0);
66          vbutton.setVisibility(0);
        }

74      public void viewCreds(View view) {
75          Toast.makeText(this, "Invalid PIN. Please try again", 0).show();
        }
    }
```

So we called the intent using adb and changed the boolean value to false which helped us to bypass the pin system.



11. Lab 9 — Access Control Part 3 (Content Provider)

Objective
Retrieve private notes directly through the Content Provider without entering the UI PIN.

Analysis
AccessControl3NotesActivity checked the PIN before displaying notes. NotesProvider.query() queried SQLite without its own PIN/authorization check.

Provider URI
content://jakhar.aseem.diva.provider.notesprovider/notes

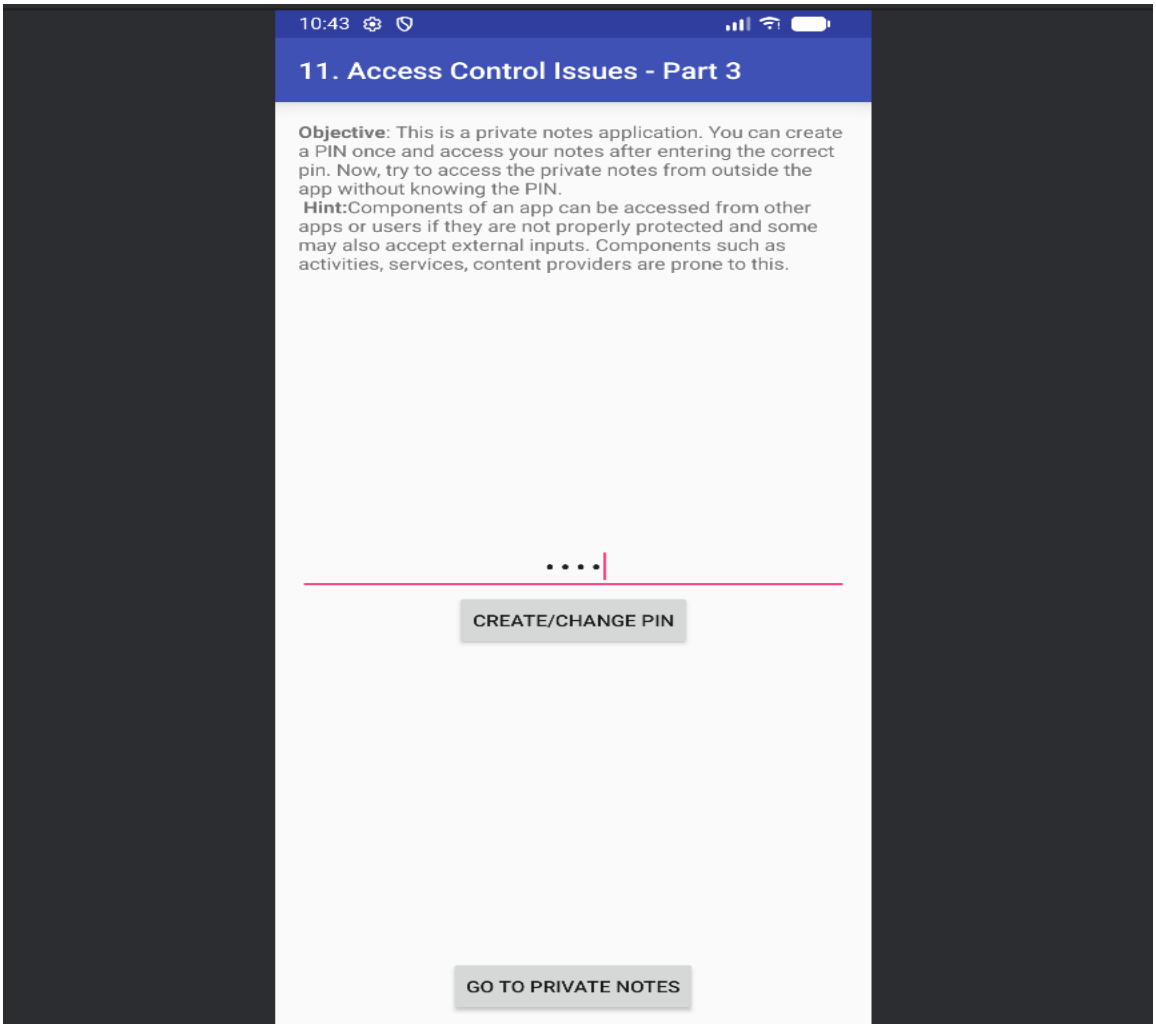
Test command
adb shell content query --uri content://jakhar.aseem.diva.provider.notesprovider/notes

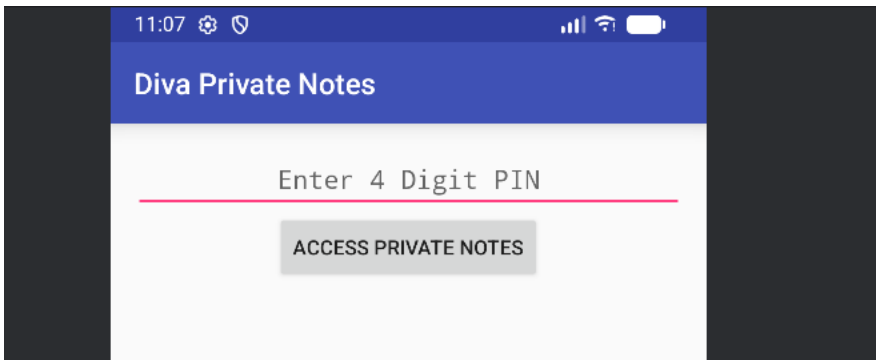
Observation
The command returned notes without entering the PIN in the app, confirming direct provider access in this lab.

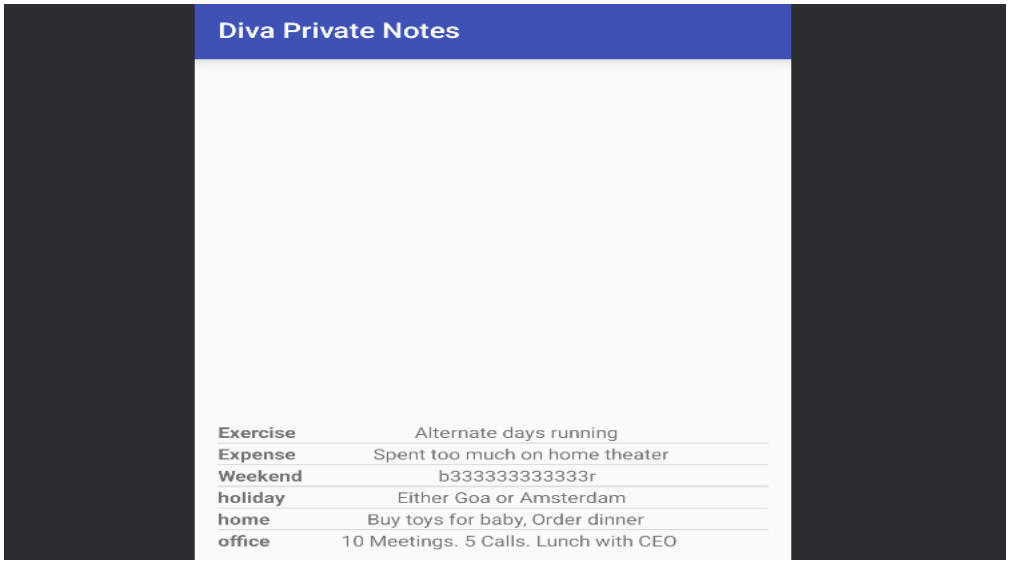
Impact and remediation
Sensitive notes were exposed. Set android:exported=false when external access is unnecessary; otherwise enforce provider permissions and authorization in the provider.

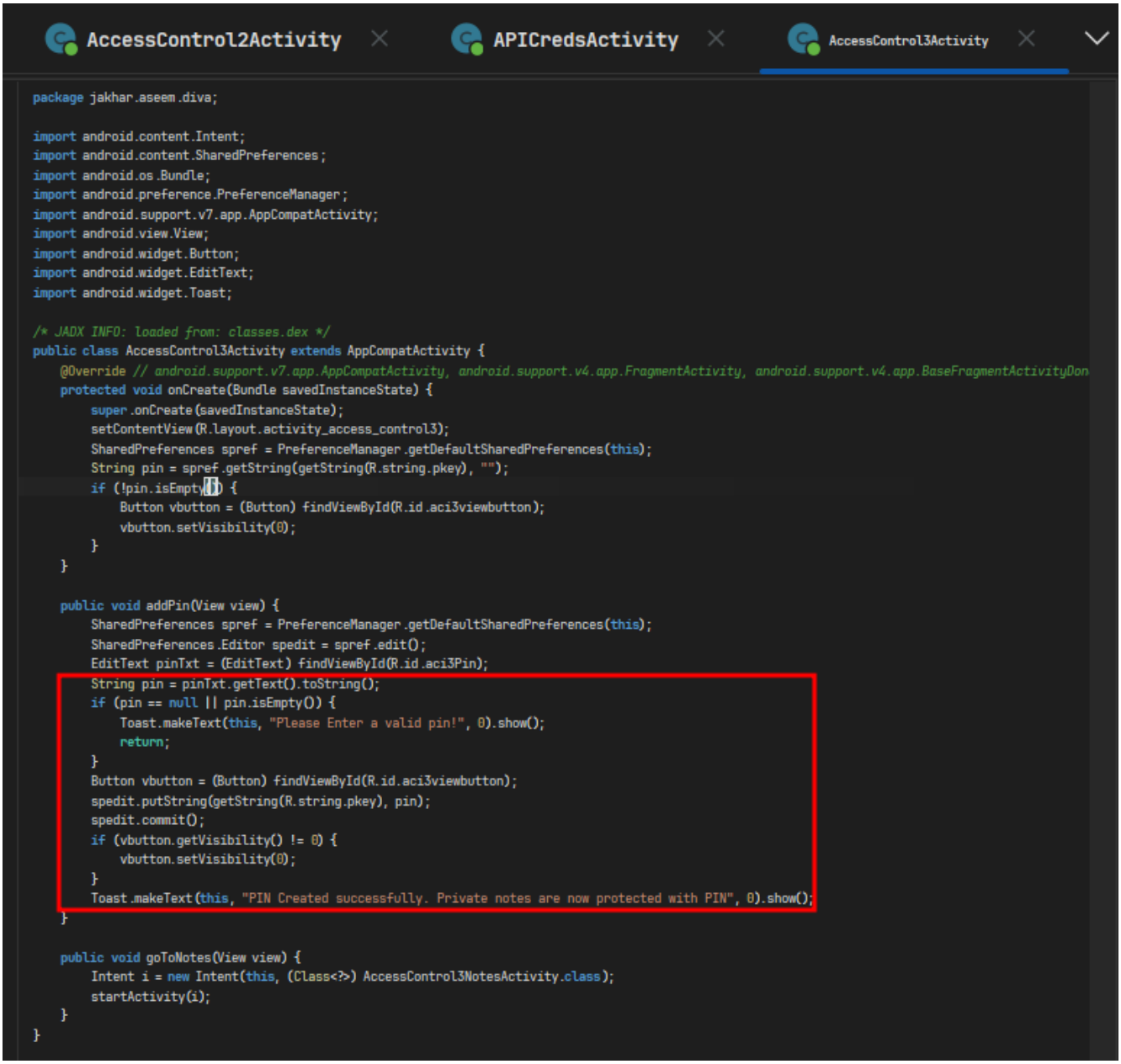
Context and learning
This exercise reinforced that a security check in the user interface does not automatically protect the underlying data. The app’s notes screen checked the PIN before querying the provider, but the Content Provider itself could be queried directly. This meant that another access path bypassed the screen’s check. I verified this using the provider URI and ADB’s content command. The protection needs to be enforced at the component or data-access boundary, not only in the Activity.

[SCREENSHOT AREA — Lab 9 — Content Provider query output]






Diva Private Notes
Exercise Alternate days running
Expense Spent too much on home theater
Weekend b333333333333r
holiday Either Goa or Amsterdam
home Buy toys for baby, Order dinner
office 10 Meetings. 5 Calls. Lunch with CEO



The PIN check was implemented in the UI Activity, but not in the Content Provider.

An attacker could skip the Activity and access the provider directly.

AccessControl3NotesActivity | NotesProvider | LogActivity

```
package jakhar.aseem.diva;

import android.content.SharedPreferences;
import android.database.Cursor;
import android.os.Bundle;
import android.preference.PreferenceManager;
import android.support.v7.app.AppCompatActivity;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ListAdapter;
import android.widget.ListView;
import android.widget.SimpleCursorAdapter;
import android.widget.Toast;

/* JADX INFO: loaded from: classes.dex */
50  public class AccessControl3NotesActivity extends AppCompatActivity {
        @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.FragmentActivity, android.support.v4.app.BaseFragmentActivityDon
51      protected void onCreate(Bundle savedInstanceState) {
52          super.onCreate(savedInstanceState);
53          setContentView(R.layout.activity_access_control3_notes);
        }

56      public void accessNotes(View view) {
57          EditText pinTxt = (EditText) findViewById(R.id.aci3notesPinText);
57          Button abutton = (Button) findViewById(R.id.aci3naccessbutton);
59          SharedPreferences spref = PreferenceManager.getDefaultSharedPreferences(this);
57          String pin = spref.getString(getString(R.string.pkey), "");
61          String userpin = pinTxt.getText().toString();
64          if (userpin.equals(pin)) {
57              ListView lview = (ListView) findViewById(R.id.aci3nlistView);
67              Cursor cr = getContentResolver().query(NotesProvider.CONTENT_URI, new String[]{"_id", "title", "note"}, null, null, null);
68              String[] columns = {"title", "note"};
69              int[] fields = {R.id.title_entry, R.id.note_entry};
70              SimpleCursorAdapter adapter = new SimpleCursorAdapter(this, R.layout.notes_entry, cr, columns, fields, 0);
71              lview.setAdapter((ListAdapter) adapter);
72              pinTxt.setVisibility(4);
73              abutton.setVisibility(4);
81              return;
            }
57          Toast.makeText(this, "Please Enter a valid pin!", 0).show();
        }
    }
```

```
[redacted]adb shell content query --uri content://jakhar.aseem.diva.provider.notesp
rovider/notes
Row: 0 _id=5, title=Exercise, note=Alternate days running
Row: 1 _id=4, title=Expense, note=Spent too much on home theater
Row: 2 _id=6, title=Weekend, note=b333333333333r
Row: 3 _id=3, title=holiday, note=Either Goa or Amsterdam
Row: 4 _id=2, title=home, note=Buy toys for baby, Order dinner
Row: 5 _id=1, title=office, note=10 Meetings. 5 Calls. Lunch with CEO
```

12. Findings Summary

Lab	Finding	Key lesson
Logging	Sensitive data in logcat	Do not log secrets
Storage 1	Plaintext SharedPreferences	Private storage is not encryption
Storage 2	Plaintext SQLite / unsafe SQL	Parameterized queries
Storage 4	Plaintext external storage	Avoid shared storage for secrets
Input 1	SQL injection	Bind SQL parameters
Input 2	Unsafe WebView URL handling	Validate schemes/hosts
Access 1	Intent Activity exposure investigated	Check export/permissions
Access 2	PIN bypass via Intent extra	Do not trust caller auth flags
Access 3	Content Provider disclosure	Protect data source

13. Tools for Future Labs

Frida: dynamic instrumentation and runtime method hooking.

Objection: mobile runtime exploration toolkit built on Frida.

Burp Suite: intercept and test HTTP/HTTPS traffic and APIs.

These tools were not used in the labs documented above and should be documented separately after practical use.

14. Conclusion

The exercises demonstrated that Android security must be assessed beyond the visible UI. Logs, local storage, SQL statements, Intents, and Content Providers can expose sensitive data or bypass UI protections. Secure apps must validate input, protect stored data, and enforce authorization at component and data-access layers.

15. Screenshot Checklist

- Lab 1 — logcat output
- Lab 2 — SharedPreferences XML
- Lab 3 — SQLite database
- Lab 4 — external storage file
- Lab 5 — SQL injection result
- Lab 6 — WebView URI behavior
- Lab 7 — Activity launch
- Lab 8 — PIN bypass result
- Lab 9 — Content Provider query output